

# Multimodal Medical Assistant for Image-Text Clinical Triage

HPPCS[04] — MedGemma 1.5 4B, Llama 3.2 and LangGraph

## Abstract

This project implements an educational Multimodal Medical Assistant that reads five synthetic image-note pairs and writes five conversation JSON files (HPPCS[04]). The general capstone brief asks for at least two large language models. This rewrite therefore uses two generative LLMs with a clear split of labour: MedGemma 1.5 4B (vision) inspects the medical teaching image together with the patient note; Llama 3.2 (text) extracts entities, writes structured triage JSON, and produces the clinician-facing dialogue. LangGraph is the orchestrator: a four-node state graph runs MedGemma first, then three Llama nodes, so each model sees only the evidence it needs. OpenCV remains only in the dataset builder that draws the synthetic PNGs. CLIP, BioBERT and LangChain LCEL chains were removed. This is an educational prototype, not a diagnostic device.

## 1. Introduction

A multimodal assistant must both look at a picture and talk about a case. Putting those jobs into one undifferentiated script hides mistakes, such as asking a small text model to invent imaging signs. The previous implementation used OpenCV, PubMedCLIP and BioBERT as encoders and a single Llama 3.2 generator. Encoders are not LLMs, so that design did not meet a two-LLM requirement. The rewrite keeps the same five teaching cases and the same JSON deliverable, but replaces the encoder stack with a medical vision LLM and keeps Llama 3.2 as the writer.

## 2. Problem Statement

Given a medical image and a short prescription-style note, produce a multi-turn conversation that summarises the case, correlates image findings with symptoms, assigns triage (emergency, urgent, soon or routine) and asks follow-up questions, with a non-clinical disclaimer. Five such conversations must be written to disk.

## 3. Objectives

- Emit five conversation JSON files from five image-note pairs.
- Use two LLMs: MedGemma 1.5 4B (vision) and Llama 3.2 (text generator).
- Use LangGraph to order image understanding, entity extraction, reasoning and dialogue.
- Keep a non-clinical educational disclaimer on every assistant turn.
- Score the synthetic set with keyword recall and triage match, stating that these are not clinical metrics.

## 4. Why these two LLMs

MedGemma 1.5 4B is a Gemma-3-based multimodal model trained for medical text and medical images (radiology, dermatology, ophthalmology and related teaching domains). It accepts image plus text and emits text. Llama 3.2 3B (or 1B on a 4 GB machine) is a general instruction-tuned text LLM. It cannot see pixels, but it is compact, already named in the original brief, and is used here only as the generator: entities, triage JSON and dialogue. The two models are not duplicates. MedGemma is not used to write the full conversation, and Llama is not asked to look at the PNG. On a small computer they are loaded one after the other (MedGemma keep\_alive=0) so both weight files are not held in RAM at once.

## 5. Why LangGraph

LangGraph is not a medical model. It records the order of work as a graph of named nodes that share a typed state. Node 1 (medgemma\_analyze) writes visual\_analysis. Node 2 (llama\_entities) writes extracted\_entities. Node 3 (llama\_reason) writes clinical\_reasoning using MedGemma findings plus the note. Node 4 (llama\_converse) writes the conversation. Edges are linear: START → MedGemma → Llama entities → Llama reasoning → Llama dialogue → END. That graph is the markable orchestration layer. Prompt text lives next to the node that uses it. JSON repair stays in the Ollama client so malformed small-model output does not crash the graph.

## 6. Methodology / Workflow

dataset\_builder.py procedurally draws five 256×256 teaching images (chest radiograph with opacity, irregular mole, brain CT with a dense wedge, fundus with blot haemorrhages, wrist radiograph with a lucent line) and writes matching de-identified notes. No real patient data is used. main.py checks that Ollama is up, resolves medgemma:4b (or an installed alias) and llama3.2:3b (falling back to llama3.2:1b). For each case, LangGraph invokes MedGemma through Ollama /api/chat with the PNG attached as base64, then invokes Llama three times for JSON entities, JSON reasoning, and prose turns. A note-based calibration step then aligns triage with explicit red flags (for example sudden hemiparesis → emergency; stable mole → soon). This calibration is a safety overlay for small generators that over-call emergency; it does not replace clinician judgement. Keyword recall and exact triage match are computed against the known synthetic labels and stored in evaluation\_summary.json.

## 7. System Design / Implementation

main.py is the entry point. graph\_pipeline.py defines CaseState, the four LangGraph nodes and prompt contracts. medical\_assistant.py compiles the graph once, packages each JSON record and aggregates metrics. ollama\_client.py wraps /api/chat and /api/tags, strips optional traces, and extracts JSON from fenced or noisy completions. dataset\_builder.py holds GROUND\_TRUTH keywords used only for educational scoring. OpenCV is not used at inference time. Removed modules: image\_analyzer.py,

multimodal\_encoder.py, langchain\_orchestrator.py.

## 8. Results and Analysis

Conversation files conversation\_01.json ... conversation\_05.json and evaluation\_summary.json are produced by python main.py after the two Ollama models are installed (ollama pull medgemma:4b and ollama pull llama3.2:3b). Metrics remain educational: keyword recall searches the whole JSON record, so words copied from the input note also count; triage accuracy can be lifted by the note-based calibration. Synthetic cartoon images are a hard vision test even for MedGemma; the note is often the stronger signal. The design goal of this rewrite is a correct two-LLM architecture with LangGraph, not a claim of clinical diagnostic accuracy.

## 9. Limitations and Ethics

The assistant must not be used for real care. Images are schematic. Small LLMs hallucinate. MedGemma on Ollama is the public 4B multimodal tag (medgemma:4b), which tracks Google's MedGemma family for medical image-text work; community quantisations are accepted as aliases if the official tag is absent. Llama 3.2 is not medically fine-tuned. Evaluation is not a reader study.

## 10. Conclusion and Future Work

The rewrite meets HPPCS[04] with two LLMs and a graph orchestrator: MedGemma 1.5 4B for image-note understanding, Llama 3.2 for generation, LangGraph for control flow. Future work includes running the same graph on de-identified real studies, replacing keyword recall with clinician-rated conversation quality, and optional Llama 3.2 Vision only if a second vision model is explicitly required.

## 11. References

[1] HAAI++ capstone instructions and HPPCS[04] brief. [2] A. Grattafiori et al., The Llama 3 herd of models, arXiv:2407.21783. [3] Google Health AI Developer Foundations, MedGemma model card, <https://developers.google.com/health-ai-developer-foundations/medgemma/model-card> [4] LangGraph documentation, <https://langchain-ai.github.io/langgraph/> [5] Ollama, <https://ollama.com/library/medgemma> [6] Meta Llama 3.2, <https://ollama.com/library/llama3.2> [7] G. Bradski, The OpenCV library, 2000 (used only to synthesise teaching images).

## Acknowledgement

The author thanks the HAAI++ faculty and the MedGemma, Llama, LangGraph and Ollama communities. All images and notes are synthetic and educational.